



**ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ**

ΕΡΓΑΣΤΗΡΙΟ

ΒΑΣΕΙΣ ΔΕΔΟΜΕΝΩΝ II

ΕΡΓΑΣΙΑ 5

**ΦΙΛΙΠΠΟΣ ΦΙΛΙΠΠΑΤΟΣ
ΑΜ:19390242**

ΕΚΠ. ΕΤΟΣ 2024-2025

1. **Συνδεθείτε** στην MySQL του συστήματός σας με όποιον από τους προαναφερόμενους τρόπους επιθυμείτε.

```
filip@SOPPILIF c:\xampp
# mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 11
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

2. Ελέγξτε αν υπάρχει ΒΔ με την ονομασία **my_accounts**.

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| personnel |
| phpmyadmin |
| test |
+-----+
6 rows in set (0.032 sec)
```

3. Δημιουργήστε τη ΒΔ my_accounts, επιλέξτε την για χρήση και δημιουργήστε και πίνακα με όνομα Accounts με δομή και περιεχόμενα, όπως φαίνονται στις ακόλουθες εντολές. Δείξτε το αποτέλεσμα εμφανίζοντας (α) τη λίστα πινάκων της ΒΔ, (β) τα περιεχόμενα και (γ) τη δομή του πίνακα Accounts.

```
drop database if exists my_accounts;
create database my_accounts;
use my_accounts;
create table Accounts ( acctID integer not null primary key,
Balance integer not null);
insert into Accounts (acctID, Balance) values (101, 1000);
insert into Accounts (acctID, Balance) values (202, 2000);
insert into Accounts (acctID, Balance) values (303, 2500);
insert into Accounts (acctID, Balance) values (404, 3000);
```

```
MariaDB [(none)]> drop database if exists my_accounts;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [(none)]> create database my_accounts;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]> use my_accounts;
Database changed
MariaDB [my_accounts]> create table Accounts ( acctID integer not null primary key,
-> Balance integer not null);
Query OK, 0 rows affected (0.011 sec)

MariaDB [my_accounts]> insert into Accounts (acctID, Balance) values (101, 1000);
Query OK, 1 row affected (0.009 sec)

MariaDB [my_accounts]> insert into Accounts (acctID, Balance) values (202, 2000);
Query OK, 1 row affected (0.007 sec)

MariaDB [my_accounts]> insert into Accounts (acctID, Balance) values (303, 2500);
Query OK, 1 row affected (0.008 sec)

MariaDB [my_accounts]> insert into Accounts (acctID, Balance) values (404, 3000);
Query OK, 1 row affected (0.007 sec)

MariaDB [my_accounts]> show tables;
+-----+
| Tables_in_my_accounts |
+-----+
| accounts               |
+-----+
1 row in set (0.001 sec)

MariaDB [my_accounts]> select * from accounts;
+-----+-----+
| acctID | Balance |
+-----+-----+
| 101    | 1000    |
| 202    | 2000    |
| 303    | 2500    |
| 404    | 3000    |
+-----+-----+
4 rows in set (0.001 sec)

MariaDB [my_accounts]> describe accounts;
+-----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| acctID | int(11) | NO   | PRI | NULL    |       |
| Balance | int(11) | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.026 sec)
```

4. Εμφανίστε τα περιεχόμενα του πίνακα Accounts, προσθέτοντας αύξουσα αρίθμηση στις εγγραφές του:

```
set @rownum=0;  
select (@rownum:=@rownum+1) as No, acctID, Balance from Accounts order by acctID;
```

```
MariaDB [my_accounts]> set @rownum=0;  
Query OK, 0 rows affected (0.000 sec)  
  
MariaDB [my_accounts]> select (@rownum:=@rownum+1) as No, acctID, Balance from Accounts order by acctID;  
+-----+-----+-----+  
| No    | acctID | Balance |  
+-----+-----+-----+  
| 1     | 101    | 1000    |  
| 2     | 202    | 2000    |  
| 3     | 303    | 2500    |  
| 4     | 404    | 3000    |  
+-----+-----+-----+  
4 rows in set (0.001 sec)
```

5. Η αύξουσα αρίθμηση που εμφανίστηκε στην στήλη με τίτλο No του βήματος 4, θα πρέπει να υπάρχει και στον πίνακα Accounts; Αιτιολογήστε την απάντησή σας.

Όχι καθώς δεν έχουμε καταχωρήσει κάποια στήλη με τίτλο 'No' στον πίνακα 'Accounts'. Στο βήμα 4 απλώς εμφανίσαμε τον πίνακα 'Accounts' προσθέτοντας προσωρινά τη στήλη 'No' με την αύξουσα αρίθμηση.

6. Προσθέστε τον πίνακα CUSTOMERS που εμφανίζεται στην [Εικόνα 1](#). Ορίστε τύπους δεδομένων των στηλών CUSTNO και CUST_NAME ως integer και varchar(30) αντίστοιχα. Δείξτε το αποτέλεσμα εμφανίζοντας (α) τη λίστα πινάκων της ΒΔ, (β) τα περιεχόμενα και (γ) τη δομή του πίνακα CUSTOMERS.

```
MariaDB [my_accounts]> create table CUSTOMERS(CUSTNO integer, CUST_NAME varchar(30));
Query OK, 0 rows affected (0.012 sec)
```

```
MariaDB [my_accounts]> insert into CUSTOMERS(CUSTNO, CUST_NAME) values
    -> (10, '101'),
    -> (20, '202');
Query OK, 2 rows affected (0.004 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
MariaDB [my_accounts]> show tables;
```

```
+-----+
| Tables_in_my_accounts |
+-----+
| accounts              |
| customers              |
+-----+
2 rows in set (0.001 sec)
```

```
MariaDB [my_accounts]> select * from CUSTOMERS;
```

```
+-----+-----+
| CUSTNO | CUST_NAME |
+-----+-----+
| 10     | 101       |
| 20     | 202       |
+-----+-----+
2 rows in set (0.006 sec)
```

```
MariaDB [my_accounts]> describe CUSTOMERS;
```

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| CUSTNO     | int(11)       | YES  |     | NULL    |       |
| CUST_NAME  | varchar(30)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.024 sec)
```

7. Στον πίνακα Accounts προσθέστε στήλη με όνομα Custno, τύπο δεδομένων integer και ορίστε την ως FK του πίνακα Accounts για τη σύνδεση των εγγραφών του με τις εγγραφές του πίνακα CUSTOMERS. Ενημερώστε τα περιεχόμενα της στήλης Custno, ώστε ο λογαριασμός με AcctID=202 να αντιστοιχεί στον κωδικό πελάτη 20 και όλοι οι υπόλοιποι λογαριασμοί να αντιστοιχούν στον κωδικό πελάτη 10. Δείξτε το αποτέλεσμα εμφανίζοντας (α) τα περιεχόμενα και (β) τη δομή του πίνακα Accounts.

```
MariaDB [my_accounts]> alter table Accounts add column Custno integer;
Query OK, 0 rows affected (0.011 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MariaDB [my_accounts]> alter table CUSTOMERS add primary key(Custno);
Query OK, 0 rows affected (0.044 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MariaDB [my_accounts]> alter table Accounts
-> add foreign key(Custno) references CUSTOMERS(CUSTNO);
Query OK, 4 rows affected (0.063 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MariaDB [my_accounts]> update Accounts set Custno=20 where AcctID=202;
Query OK, 1 row affected (0.003 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
MariaDB [my_accounts]> update Accounts set Custno=10 where AcctID!=202;
Query OK, 3 rows affected (0.011 sec)
Rows matched: 3 Changed: 3 Warnings: 0
```

```
MariaDB [my_accounts]> select * from Accounts;
```

acctID	Balance	Custno
101	1000	10
202	2000	20
303	2500	10
404	3000	10

4 rows in set (0.001 sec)

```
MariaDB [my_accounts]> describe Accounts;
```

Field	Type	Null	Key	Default	Extra
acctID	int(11)	NO	PRI	NULL	
Balance	int(11)	NO		NULL	
Custno	int(11)	YES	MUL	NULL	

3 rows in set (0.024 sec)

8. Εκτελέστε και ερμηνεύστε τις ακόλουθες δηλώσεις SQL:

```
select CUSTNO, count(*), sum(Balance)
from Accounts
where CUSTNO not in (20)
group by CUSTNO;

//Παραλλαγή με χρήση μεταβλητής
set @CUST_NO=20;

select CUSTNO, count(*), sum(Balance)
from Accounts
where CUSTNO not in (@CUST_NO)
group by CUSTNO;
```

Το αποτέλεσμα είναι το ίδιο καθώς δίνουμε την ίδια τιμή (το 20) στην εντολή 'where CUSTNO not in...', απλά την δεύτερη φορά χρησιμοποιούμε την μεταβλητή '@CUST_NO' στην οποία έχουμε θέσει την τιμή '20'.

```
MariaDB [my_accounts]> select CUSTNO, count(*), sum(Balance)
-> from Accounts
-> where CUSTNO not in (20)
-> group by CUSTNO;
```

CUSTNO	count(*)	sum(Balance)
10	3	6500

1 row in set (0.001 sec)

```
MariaDB [my_accounts]> set @CUST_NO=20;
Query OK, 0 rows affected (0.000 sec)
```

```
MariaDB [my_accounts]> select CUSTNO, count(*), sum(Balance)
-> from Accounts
-> where CUSTNO not in (@CUST_NO)
-> group by CUSTNO;
```

CUSTNO	count(*)	sum(Balance)
10	3	6500

1 row in set (0.001 sec)

9. Εκτελέστε και ερμηνεύστε τις ακόλουθες δηλώσεις SQL:

```
select count(*), sum(Balance) from Accounts;

//Παραλλαγή με χρήση μεταβλητών
set @COUNT_acctID=0, @SUM_acctID=0, @AVG_acctID=0;

select count(*), sum(Balance), avg(Balance)
into @COUNT_acctID, @SUM_acctID, @AVG_acctID
from Accounts;

select @COUNT_acctID, @SUM_acctID, @AVG_acctID, @MY_AVG := @SUM_acctID/@COUNT_acctID;
```

Η διαφορά είναι ότι στο δεύτερο μέρος αποθηκεύουμε το αποτέλεσμα των count(*), sum(Balance) σε μεταβλητές. Επιπλέον αποθηκεύουμε και το αποτέλεσμα της avg(Balance). Τέλος εμφανίζουμε τις τιμές των προηγούμενων μεταβλητών μαζί με την @MY_AVG, όπου υπολογίσαμε μόνοι μας τον μέσο όρο(διαιρώντας το συνολικό ποσό @SUM_acctID με τον αριθμό λογαριασμών @COUNT_acctID) και βλέπουμε ότι το αποτέλεσμα είναι ίδιο με αυτό της συνάρτησης 'avg' της MySQL.

```
MariaDB [my_accounts]> select count(*), sum(Balance) from Accounts;
+-----+-----+
| count(*) | sum(Balance) |
+-----+-----+
|          4 |          8500 |
+-----+-----+
1 row in set (0.001 sec)

MariaDB [my_accounts]> set @COUNT_acctID=0, @SUM_acctID=0, @AVG_acctID=0;
Query OK, 0 rows affected (0.000 sec)

MariaDB [my_accounts]> select count(*), sum(Balance), avg(Balance)
-> into @COUNT_acctID, @SUM_acctID, @AVG_acctID
-> from Accounts;
Query OK, 1 row affected (0.001 sec)

MariaDB [my_accounts]> select @COUNT_acctID, @SUM_acctID, @AVG_acctID, @MY_AVG := @SUM_acctID/@COUNT_acctID;
+-----+-----+-----+-----+
| @COUNT_acctID | @SUM_acctID | @AVG_acctID | @MY_AVG := @SUM_acctID/@COUNT_acctID |
+-----+-----+-----+-----+
|          4 |          8500 | 2125.0000000000 | 2125.0000000000 |
+-----+-----+-----+-----+
1 row in set (0.001 sec)
```


10. Στον πίνακα Accounts προσθέστε στήλη με όνομα Amount, τύπο δεδομένων integer και ορίστε την ως FK του πίνακα Accounts για τη σύνδεση των εγγραφών του με τις εγγραφές του πίνακα CUSTOMERS. Ενημερώστε τα περιεχόμενα της στήλης Custno, ώστε ο λογαριασμός με AcctID=202 να αντιστοιχεί στον κωδικό πελάτη 20 και όλοι οι υπόλοιποι λογαριασμοί να αντιστοιχούν στον κωδικό πελάτη 10. Δείξτε το αποτέλεσμα εμφανίζοντας (α) τα περιεχόμενα και (β) τη δομή του πίνακα Accounts.

```
MariaDB [my_accounts]> alter table Accounts add column Amount integer,
-> add foreign key(Amount) references CUSTOMERS(CUSTNO);
Query OK, 4 rows affected (0.040 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [my_accounts]> update Accounts set Custno=20 where AcctID=202;
Query OK, 0 rows affected (0.002 sec)
Rows matched: 1 Changed: 0 Warnings: 0

MariaDB [my_accounts]> update Accounts set Custno=10 where AcctID!=202;
Query OK, 0 rows affected (0.000 sec)
Rows matched: 3 Changed: 0 Warnings: 0

MariaDB [my_accounts]> select * from Accounts;
+-----+-----+-----+-----+
| acctID | Balance | Custno | Amount |
+-----+-----+-----+-----+
| 101 | 1000 | 10 | NULL |
| 202 | 2000 | 20 | NULL |
| 303 | 2500 | 10 | NULL |
| 404 | 3000 | 10 | NULL |
+-----+-----+-----+-----+
4 rows in set (0.000 sec)

MariaDB [my_accounts]> describe Accounts;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| acctID | int(11) | NO | PRI | NULL | |
| Balance | int(11) | NO | | NULL | |
| Custno | int(11) | YES | MUL | NULL | |
| Amount | int(11) | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.013 sec)
```

11. Εκτελέστε και ερμηνεύστε τις ακόλουθες δηλώσεις SQL:

```
create trigger CALC_SUM
before insert on Accounts
for each row
    set @SUM = @SUM + new.Amount;

// Δοκιμή της λειτουργίας του trigger
set @SUM = 0;
insert into Accounts values(100,1000.50),(101, 2000.50),(102, 1500.00);
select @SUM as 'TOTAL AMOUNT';
select * from Accounts;
```

Αρχικά δημιουργούμε το 'trigger CALC_SUM' το οποίο προσθέτει το ποσό της νέας εγγραφής 'NEW.Amount' στην μεταβλητή '@SUM'. Έπειτα αρχικοποιούμε την μεταβλητή '@SUM', θέτουμε την τιμή της 0 και εισάγουμε τα δεδομένα. Τέλος εμφανίζουμε το συνολικό ποσό με την '@SUM' και τον πίνακα 'Accounts'

```
MariaDB [my_accounts]> create trigger CALC_SUM
-> before insert on Accounts
-> for each row
->    set @SUM = @SUM + new.Amount;
Query OK, 0 rows affected (0.007 sec)

MariaDB [my_accounts]> set @SUM = 0;
Query OK, 0 rows affected (0.000 sec)

MariaDB [my_accounts]> insert into Accounts values(100,1000.50,10,1000.50),(101, 2000
.50,10,2000.50),(102, 1500.00,10,1500.00);
Query OK, 3 rows affected (0.003 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [my_accounts]> select @SUM as 'TOTAL AMOUNT';
+-----+
| TOTAL AMOUNT |
+-----+
|          4502 |
+-----+
1 row in set (0.000 sec)

MariaDB [my_accounts]> select * from Accounts;
+-----+-----+-----+-----+
| acctID | Balance | Custno | Amount |
+-----+-----+-----+-----+
|      100 |      1001 |      10 |      1001 |
|      101 |      2001 |      10 |      2001 |
|      102 |      1500 |      10 |      1500 |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

12. Ορίστε και χρησιμοποιήστε τη συνάρτηση factorial που υπολογίζει το $n! = 1 * 2 * \dots * n$:

```
drop function if exists factorial;
delimiter !
create function factorial(N int)
returns int
deterministic
begin
declare F int default 1;
while N > 0 do
    set F = N * F;
    set N = N - 1;
end while;
return F;
end !
delimiter ;
```

```
// Δοκιμές της λειτουργίας του trigger για N=4 και N=15
select factorial(4);

select factorial(15);
```

```
MariaDB [my_accounts]> drop function if exists factorial;
Query OK, 0 rows affected, 1 warning (0.016 sec)
```

```
MariaDB [my_accounts]> delimiter !
MariaDB [my_accounts]> create function factorial(N int)
-> returns int
-> deterministic
-> begin
-> declare F int default 1;
-> while N > 0 do
->   set F = N * F;
->   set N = N - 1;
-> end while;
-> return F;
-> end !
Query OK, 0 rows affected (0.005 sec)
```

```
MariaDB [my_accounts]> delimiter ;
MariaDB [my_accounts]> select factorial(4);
```

```
+-----+
| factorial(4) |
+-----+
|          24 |
+-----+
1 row in set (0.001 sec)
```

```
MariaDB [my_accounts]> select factorial(15);
```

```
+-----+
| factorial(15) |
+-----+
|    2147483647 |
+-----+
1 row in set (0.001 sec)
```

13. Ορίστε και χρησιμοποιήστε την διαδικασία `my_procedure_Local_Variables` για υπολογισμούς με χρήση τοπικών μεταβλητών:

```
drop procedure my_procedure_Local_Variables;
delimiter $$
create procedure my_procedure_Local_Variables()
begin
set @X = 25;
set @Y = 10;
select @X, @Y, @X*@Y;
end $$
delimiter ;
// Κλήση της διαδικασίας
call my_procedure_Local_Variables();
```

```
MariaDB [my_accounts]> drop procedure my_procedure_Local_Variables;
ERROR 1305 (42000): PROCEDURE my_accounts.my_procedure_Local_Variables does not exist
MariaDB [my_accounts]> delimiter $$
MariaDB [my_accounts]> create procedure my_procedure_Local_Variables()
-> begin
-> set @X = 25;
-> set @Y = 10;
-> select @X, @Y, @X*@Y;
-> end $$
Query OK, 0 rows affected (0.003 sec)

MariaDB [my_accounts]> delimiter ;
MariaDB [my_accounts]> call my_procedure_Local_Variables();
+-----+-----+-----+
| @X    | @Y    | @X*@Y |
+-----+-----+-----+
| 25    | 10    | 250   |
+-----+-----+-----+
1 row in set (0.000 sec)

Query OK, 0 rows affected (0.001 sec)
```

14. Ακολουθήστε τα ακόλουθα για τη δημιουργία μιας αποθηκευμένης διαδικασίας και χρήση των commit/rollback. Εξηγείστε τι κάνει η διαδικασία myProc.

```
// Δοκιμές χρήσης της συνάρτησης MOD
SET @p_no=3;
SELECT MOD(@p_no,2);
SET @p_no=8;
SELECT MOD(@p_no,2);

// Δημιουργία βάσης και πίνακα
DROP TABLE IF EXISTS myTrace;
CREATE TABLE myTrace ( t_no INT,
t_user CHAR(20),
t_date DATE,
t_time TIME,
t_proc VARCHAR(16), t_what VARCHAR(30));

// Δημιουργία αποθηκευμένης διαδικασίας myProc
DROP PROCEDURE IF EXISTS myProc;
DELIMITER !
CREATE PROCEDURE myProc (IN p_no INT, IN p_in VARCHAR(30),
OUT p_out VARCHAR(30))
LANGUAGE SQL
BEGIN
SET p_out=p_in;
INSERT INTO myTrace (t_no, t_user, t_date, t_time, t_proc, t_what)
VALUES (p_no, current_user, current_date, current_time, 'myProc', p_in);
IF (MOD(p_no,2)=0) THEN
COMMIT;
ELSE ROLLBACK;
END IF;
END !
DELIMITER ;

// Κλήση της διαδικασίας
SET AUTOCOMMIT=0;
CALL myProc(1, 'hello1', @out);
CALL myProc(2, 'hello2', @out);
CALL myProc(3, 'hello3', @out);
CALL myProc(4, 'hello4', @out);
CALL myProc(5, 'hello5', @out);
CALL myProc(6, 'hello6', @out);
CALL myProc(7, 'hello7', @out);

// Μελέτη αποτελέσματος
SELECT * FROM myTrace;
```

Η διαδικασία λειτουργεί ως ένα σύστημα καταγραφής που διατηρεί μόνο τις εγγραφές με άρτιο αριθμό και απορρίπτει τις εγγραφές με περιττό αριθμό (rollback). Αρχικά δέχεται τρεις παραμέτρους 'IN p_no', 'IN p_in' και 'OUT p_out'. Μετά αντιγράφει το 'p_in' στο 'p_out' και εισάγει στον πίνακα myTrace μια εγγραφή με τα '(p_no, current_user, current_date, current_time, 'myProc', p_in)'. Έπειτα ελέγχει αν το p_no είναι άρτιος ή περιττός όπου αν είναι άρτιος (MOD(p_no,2)=0) κάνει COMMIT, ενώ αν είναι περιττός κάνει ROLLBACK. Στην κλήση της διαδικασίας το AUTOCOMMIT=0 απενεργοποιεί το αυτόματο commit και μετά γίνονται 7 κλήσεις με διαφορετικά νούμερα και strings. Τέλος το αποτέλεσμα στον πίνακα myTrace περιέχει μόνο τις εγγραφές με άρτιο p_no (2,4,6) καθώς οι εγγραφές με

περιπτώ p_no (1,3,5,7) έχουν κάνει rollback.

```
MariaDB [my_accounts]> SET @p_no=3;
Query OK, 0 rows affected (0.000 sec)

MariaDB [my_accounts]> SELECT MOD(@p_no,2);
+-----+
| MOD(@p_no,2) |
+-----+
|             1 |
+-----+
1 row in set (0.000 sec)

MariaDB [my_accounts]> SET @p_no=8;
Query OK, 0 rows affected (0.000 sec)

MariaDB [my_accounts]> SELECT MOD(@p_no,2);
+-----+
| MOD(@p_no,2) |
+-----+
|             0 |
+-----+
1 row in set (0.000 sec)

MariaDB [my_accounts]> DROP TABLE IF EXISTS myTrace;
Query OK, 0 rows affected, 1 warning (0.000 sec)

MariaDB [my_accounts]> CREATE TABLE myTrace ( t_no INT,
-> t_user CHAR(20),
-> t_date DATE,
-> t_time TIME,
-> t_proc VARCHAR(16), t_what VARCHAR(30));
Query OK, 0 rows affected (0.009 sec)

MariaDB [my_accounts]> DROP PROCEDURE IF EXISTS myProc;
Query OK, 0 rows affected, 1 warning (0.000 sec)

MariaDB [my_accounts]> DELIMITER !
MariaDB [my_accounts]> CREATE PROCEDURE myProc (IN p_no INT, IN p_in VARCHAR(30),
-> OUT p_out VARCHAR(30))
-> LANGUAGE SQL
-> BEGIN
-> SET p_out=p_in;
-> INSERT INTO myTrace (t_no, t_user, t_date, t_time, t_proc, t_what)
-> VALUES (p_no, current_user, current_date, current_time, 'myProc', p_in);
-> IF (MOD(p_no,2)=0) THEN
-> COMMIT;
-> ELSE ROLLBACK;
-> END IF;
-> END !
Query OK, 0 rows affected (0.003 sec)
```

```

MariaDB [my_accounts]> DELIMITER ;
MariaDB [my_accounts]> SET AUTOCOMMIT=0;
Query OK, 0 rows affected (0.001 sec)

MariaDB [my_accounts]> CALL myProc(1, 'hello1', @out);
Query OK, 1 row affected (0.007 sec)

MariaDB [my_accounts]> CALL myProc(2, 'hello2', @out);
Query OK, 1 row affected (0.004 sec)

MariaDB [my_accounts]> CALL myProc(3, 'hello3', @out);
Query OK, 1 row affected (0.001 sec)

MariaDB [my_accounts]> CALL myProc(4, 'hello4', @out);
Query OK, 1 row affected (0.001 sec)

MariaDB [my_accounts]> CALL myProc(5, 'hello5', @out);
Query OK, 1 row affected (0.003 sec)

MariaDB [my_accounts]> CALL myProc(6, 'hello6', @out);
Query OK, 1 row affected (0.003 sec)

MariaDB [my_accounts]> CALL myProc(7, 'hello7', @out);
Query OK, 1 row affected (0.004 sec)

MariaDB [my_accounts]> SELECT * FROM myTrace;
+-----+-----+-----+-----+-----+-----+
| t_no | t_user      | t_date   | t_time    | t_proc | t_what |
+-----+-----+-----+-----+-----+-----+
| 2    | root@localhost | 2025-01-17 | 20:49:23 | myProc | hello2 |
| 4    | root@localhost | 2025-01-17 | 20:49:32 | myProc | hello4 |
| 6    | root@localhost | 2025-01-17 | 20:49:37 | myProc | hello6 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

```

Τα υπόλοιπα ερωτήματα δεν ζητάνε κάποια επεξήγηση ή output οπότε εδώ τελειώνει η 'Εργασία 5'.